Design & Analysis of Algorithms
UNIT -2

ANALYSIS OF ALGORITHM & COMPLEXITY THEORY

Prepared By Mr. Vipin K. Wani

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)



WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b



Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

Asymptotic Notations

- Asymptotic notations are mathematical tools to represent time complexity of algorithms for asymptotic analysis.
- The main idea of asymptotic analysis is to have a measure of efficiency of algorithms that doesn't depend on machine specific constants, and doesn't require algorithms to be executed and time taken by programs to be compared.
- The following 3 asymptotic notations are mostly used to represent time complexity of algorithms.

1. Big Oh Notation (O):
2. Omega Notation (Ω):
3. Theta Notation (Θ):

TG:sppu_te_be-comp

Asymptotic Notations

Big Oh Notation (O): The Big O notation defines an upper bound of an algorithm, it indicates highest possible (worst) value of time complexity .

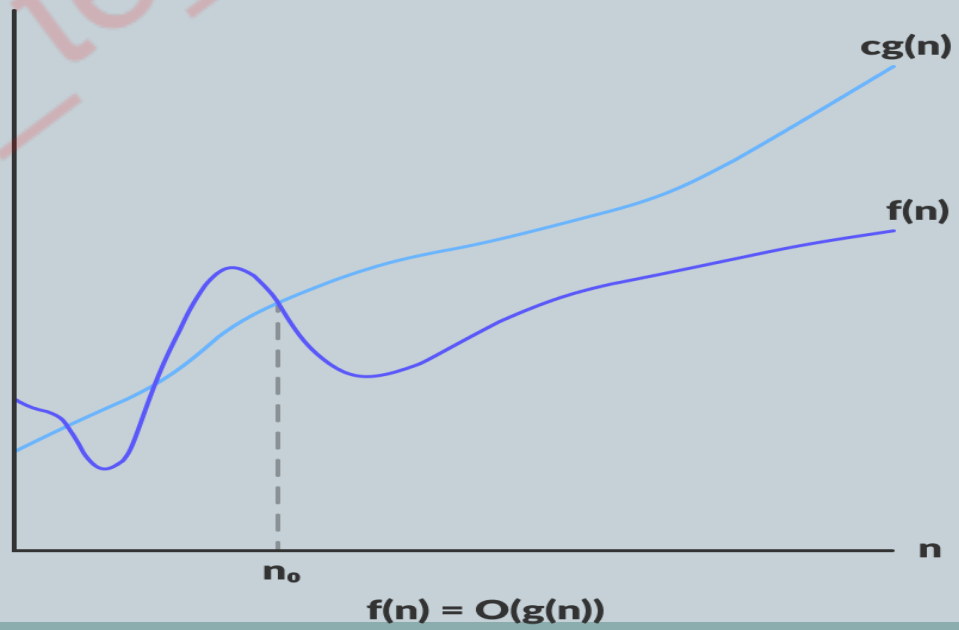
Consider a input function f(n) belongs to the set O(g(n)) if there exists a positive constant c such that it lies between 1 and g(n), for sufficiently large n i.e. $0 < cg(n) < n$. Then

$$f(n) <= C\ g(n)$$

Then we can say

$$f(n) = O\ g(n)$$

It can be represented on graph as.



TC:sppu_te_be-comp

Asymptotic Notations

Omega Notation (Ω): The Omega notation defines an lower bound of an algorithm, it indicates lowest possible (best) value of time complexity .

if there exists a positive constant c such that it lies above cg(n), for sufficiently large n i.e. cg(n)<C< n.

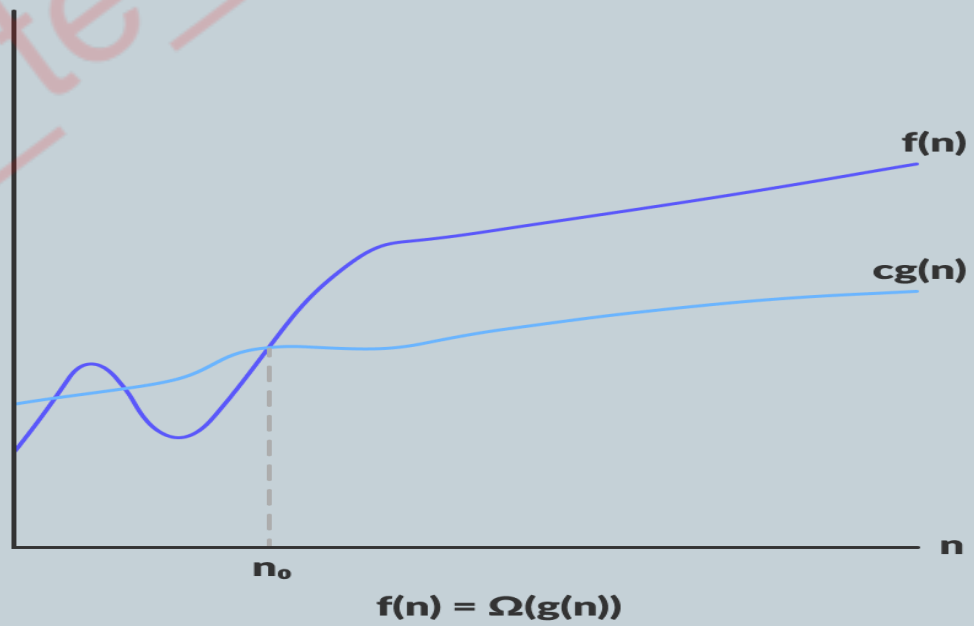
Then

$$f(n) > C\ g(n)$$

Then we can say

$$f(n) = \Omega\ g(n)$$

It can be represented on graph as.



TG:sppu_te_be_comp

Asymptotic Notations

Omega Notation (Ω): The Omega notation defines an lower bound of an algorithm, it indicates lowest possible (best) value of time complexity .

if there exists a positive constant c such that it lies above cg(n), for sufficiently large n i.e. cg(n)<C< n.

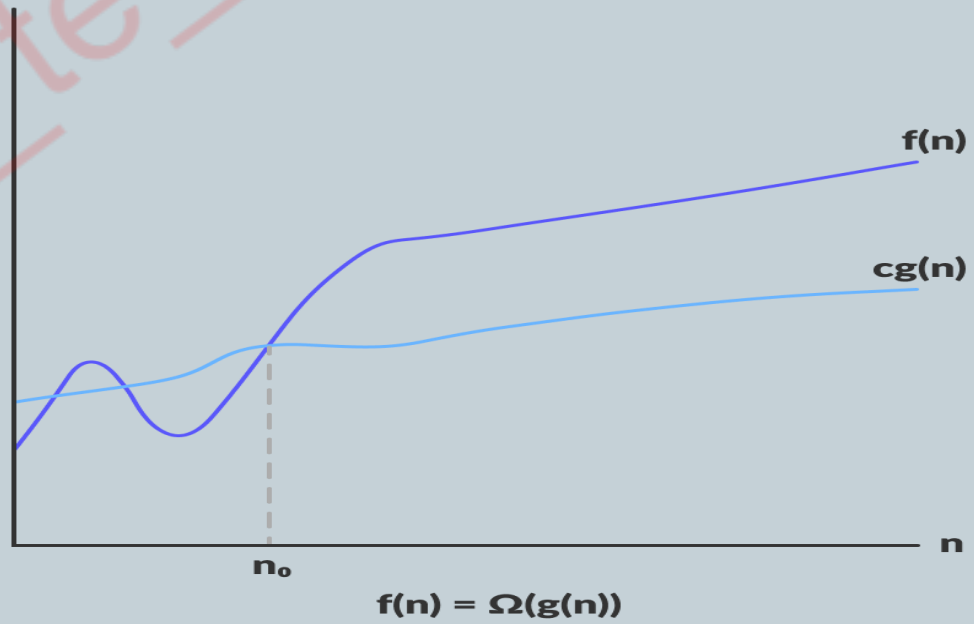
Then

$$f(n)>C\ g(n)$$

Then we can say

$$f(n)= \Omega\ g(n)$$

It can be represented on graph as.



TG:sppu_te_be_comp

Asymptotic Notations

Theta Notation (Θ): Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average case complexity of an algorithm.

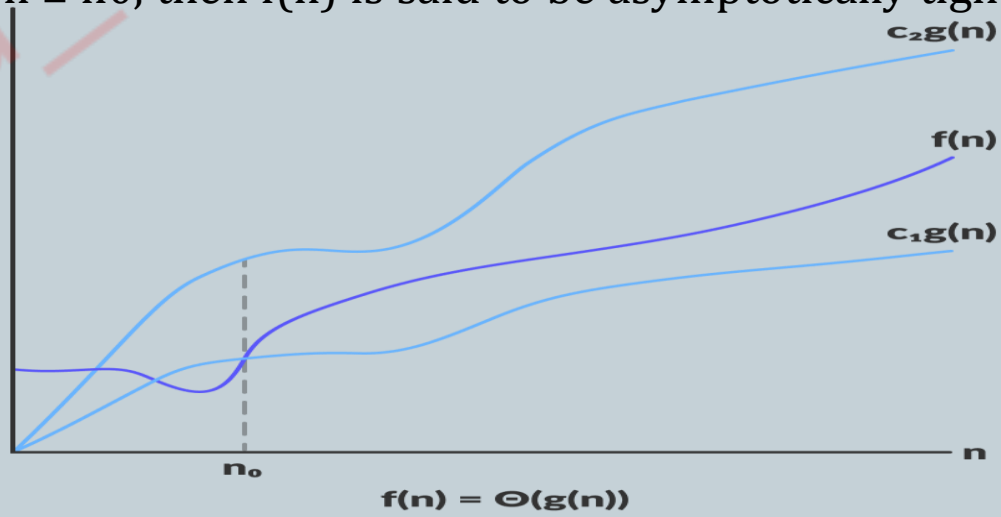
if there exist positive constants c_1 and c_2 such that it can be sandwiched between $c_1 g(n)$ and $c_2 g(n)$, for sufficiently large n. If a function f(n) lies anywhere in between $c_1 g(n)$ and $c_2 > g(n)$ for all $n \geq n0$, then f(n) is said to be asymptotically tight bound.

$$C1\ g(n) < f(n) < C2\ g(n)$$

Then we can say

$$f(n) = \Theta\ g(n)$$

It can be represented on graph as.



TG:sppu_te_be-comp

Best Case, Worst Case & Average Case Analysis

The Best Case, Worst Case & Average Case analysis is to have a measure of efficiency of algorithms

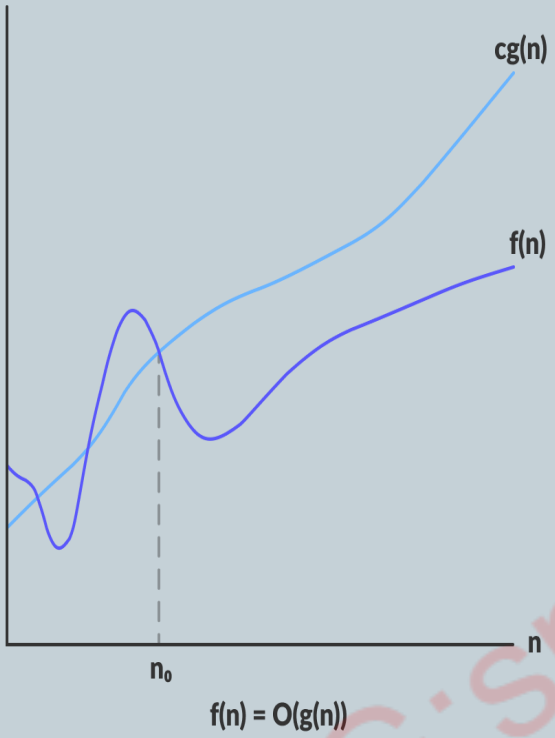
Best Case Analysis : In the best case analysis, we calculate lower bound on running time of an algorithm. We must know the case that causes minimum number of operations to be executed. In the linear search problem, the best case occurs when x is present at the first location. So time complexity in the best case would be $\Omega(1)$

Worst Case Analysis: In the worst case analysis, we calculate upper bound on running time of an algorithm. We must know the case that causes maximum number of operations to be executed. For Linear Search, the worst case happens when the element to be searched is not present in the array.

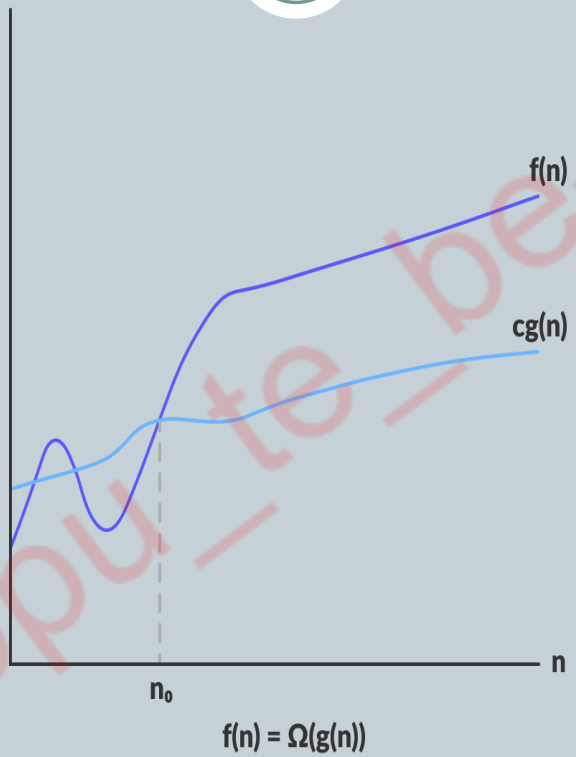
Average Case Analysis: In average case analysis, we take all possible inputs and calculate computing time for all of the inputs. Sum all the calculated values and divide the sum by total number of inputs. We must know (or predict) distribution of cases.

TG:sppu_te_be-comp

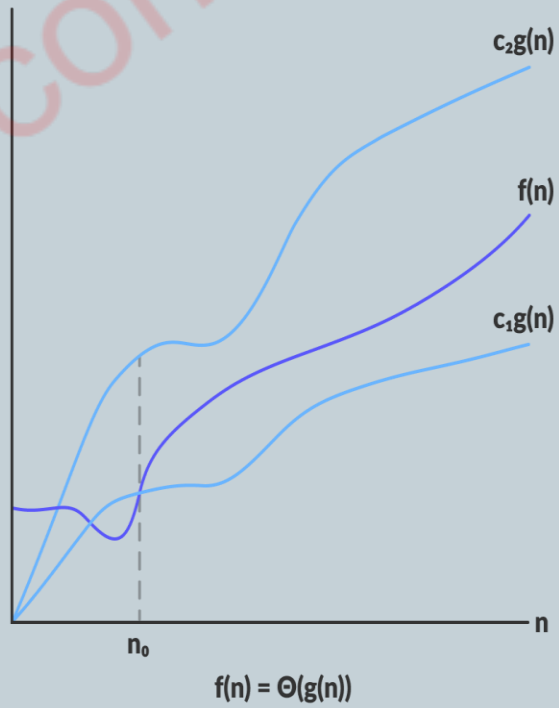
Best Case, Worst Case & Average Case Analysis



Best Case



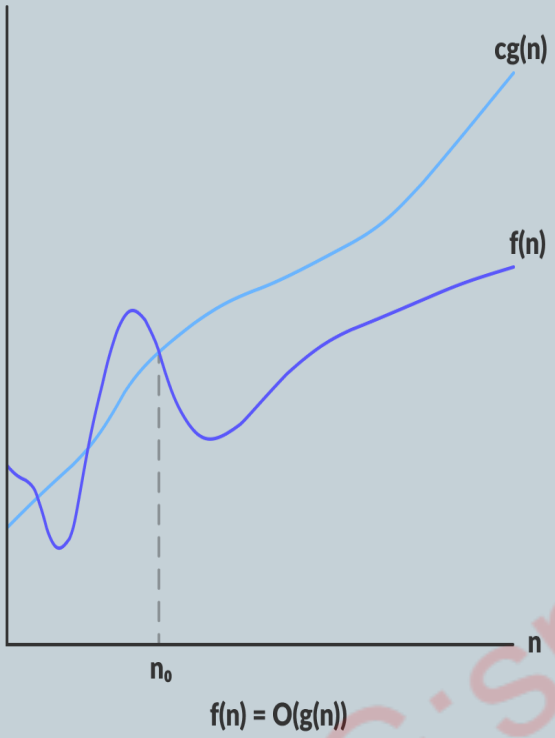
Worst Case



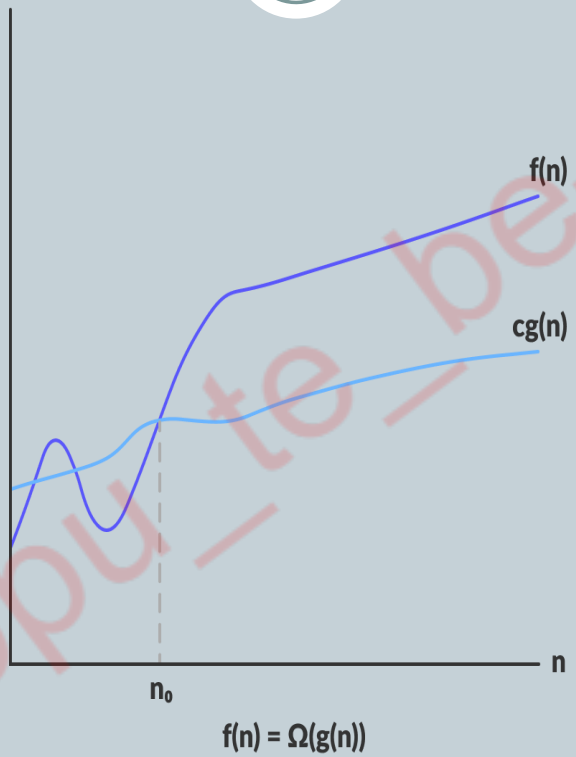
Average Case

TG:sppu_te_be-comp

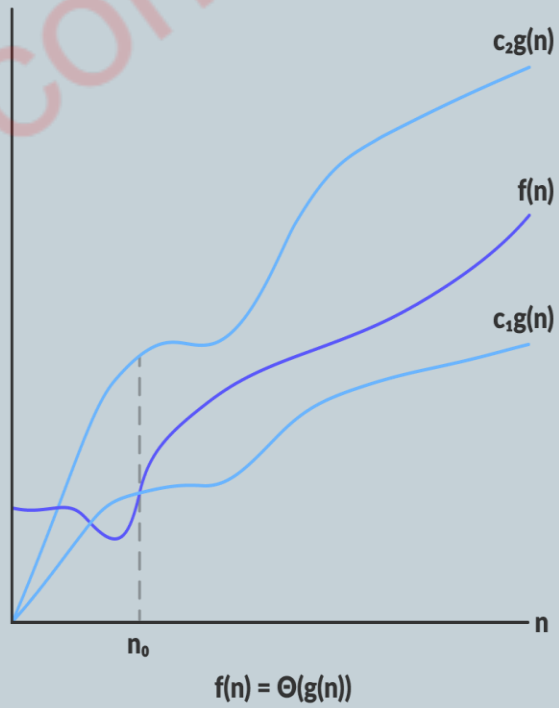
Best Case, Worst Case & Average Case Analysis



Best Case



Worst Case



Average Case

TG:sppu_te_be-comp

Input Size in Complexity

- We define input size as **the total number of items present in the input.** If we increase the input size, the total number of operations performed by an algorithm will increase.
- In other words, the time taken by an algorithm will increase with the growth in the input size.

Example:

- Sorting – The number of items to be sorted.
- Graphs – The number of vertices and/or edges.
- Numerical – The number of bits needed to represent a number

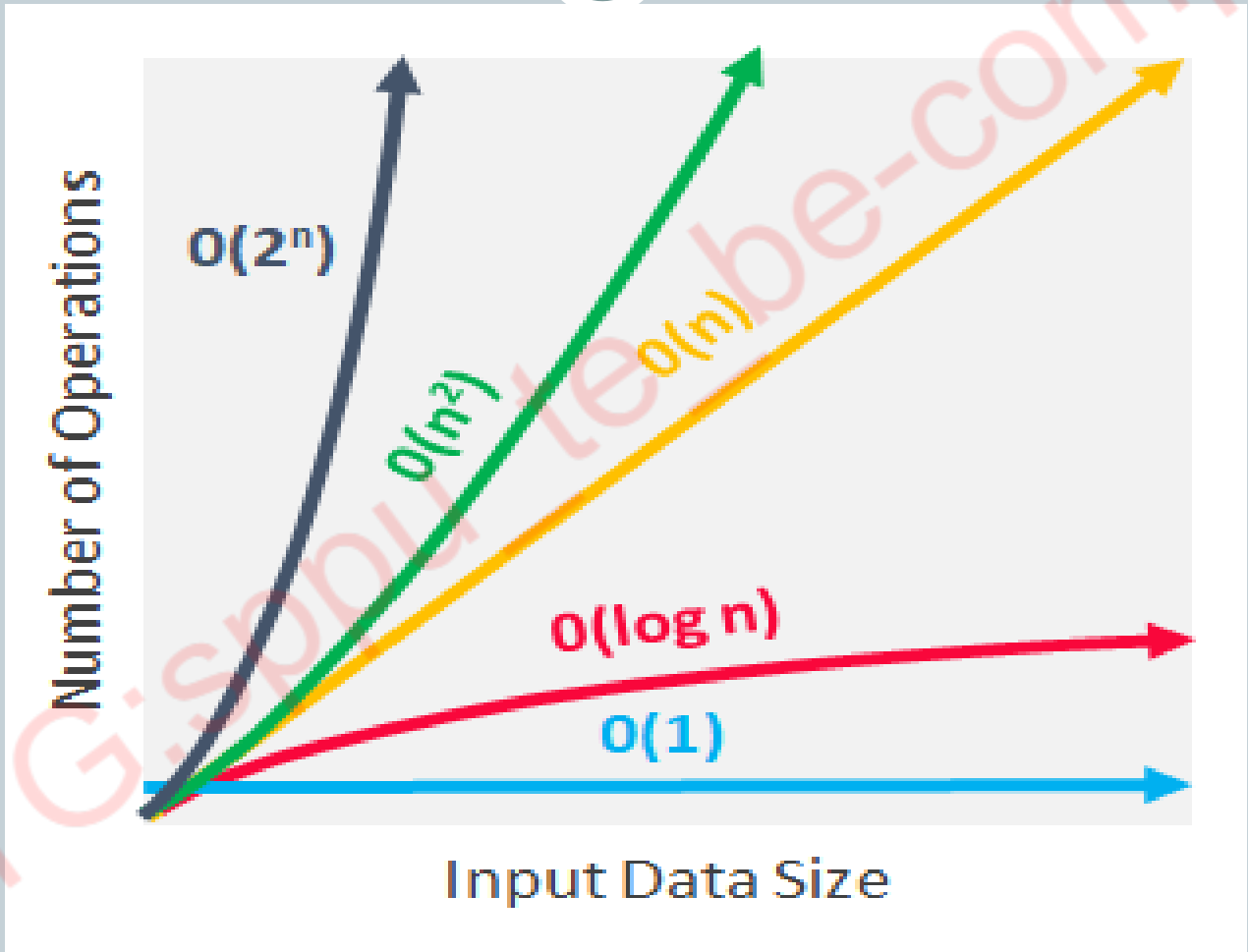
TG:sppu_to_be_comp

Growth Rate

- Algorithms analysis is all about understanding growth rates.
- That is as the amount of data gets bigger, how much more resource will my algorithm require? Typically, we describe the resource growth rate of a piece of code in terms of a function.
- To help understand the implications, this section will look at graphs for different growth rates from most efficent to least efficient.

TG:sppu_te_be_comp

Growth Rate with Diffrent input



TG:sppu_te_be-comp

Asymptotic Growth

Asymptotic Growth means the slower growth rate, and the better the algorithm.

By this measure, a linear algorithm is always asymptotically better than a quadratic one.

Ex. Algorithms having comexity

logarithmic : **O(log n):**

linear **O(n):**

TG:sppu_te_be-comp

Little o Notation

Big-O is used as a can not be tight or loose upper bound on the growth of an algorithm's effort.

"Little-o" (o()) notation is used to describe an upper bound that is tight upper bound.

Little o is a rough estimate of the maximum order of growth whereas Big-O may be the actual order of growth.

F(t) = o (g(t)) iff $\lim_{t \to \infty} \frac{f(t)}{g(t)} = 0.$

F(n) =o(g(n)) for any Constant C>o if n_o>o

i.e. F(n) =o(g(n)) F(n) < C(g(n))

TG:sppu_te_be-comp

Little w Notation

It is Used to indicate lower bound that is asymptotically tight.

F(t) = w (g(t)) iff $\lim_{t \to \infty} \frac{f(t)}{g(t)} = \infty.$

F(n) =w(g(n)) for any Constant C>0 if n_0>0

i.e. F(n) =w(g(n)) iff F(n) > C(g(n))

TG:sppu_te_be-comp

Recurrence Relation

A recurrence relation is **an equation which represents a sequence based on some rule**. It helps in finding the subsequent term (next term) dependent upon the preceding term (previous term). If we know the previous term in a given series, then we can easily determine the next term.

TG:sppu_te_be-comp

Types of Recurrence Relation

1. Homogeneous & non Homogenous

   Ax+bx+c=f(n) is a Homogeneous iff f(n)=0

   Else is Non Homogenous

2 Linear & non linear

   T(n) = f(n) +c is a Linear

   T(n)=T(n-1)+1 is a Non Linear

TG:spu_te_be-comp

How to Solve Recurrence Relation

There are 3 methods to Solving the RR

1. Substitution method
    a. Forward
    b backward
2. Master Theorem
3. Tree method

TG:sppu_te_be-comp

Recurrence Relation (Example)

1. Solve the following Recurrance Relations T(n) = T (n-1) +n if T(0)= 0

By forward substitutions

If n=1

T(1)=T(0)+1 =1

If n=2

T(2)=T(1)+2 =1+2=3

If n=3

T(3)=T(2)+3 =3+3=6

So from above values determine the some common pattern

n(n+1)/2 = (n*n +n)/2 = n*n/2 +n/2 == O(n*n)

TG:sppu_te_be-comp

Recurrence Relation (Example)

1. Solve the following Recurrance Relations $T(n) = T(n-1) + n$ if $T(0) = 0$ **By Backward Substitution.**

Solution:

$T(n) = T(n-1) + n$ ----------Eq. 1

Now here to compute T(n) we require T(n-1) so for that Replace n by n-1 in eq 1

$T(n-1)=T(n-1-1) + (n-1)$

$T(n-1) =T(n-2)+ (n-1)$

Put back the value of T(n-1) in eq 1 so it will become

$T(n)= T(n-2)+(n-1)+n$ ----------2

Now here to compute T(n) we require T(n-2) so for that Replace n by n-2 in eq 1

$T(n-2)=T(n-2-1) + (n-2)$

$T(n-2)=T(n-3) + (n-2)$

TG:sppu_te_be-comp

Recurrence Relation (Example)

Put back the value of T(n-2) in eq 2 so it will become

$T(n)= T(n-3) + (n-2)+(n-1)+n$

If we continue this replacement till K then kth term will be

$T(n)=T(n-k)+ (n-(k-1))+((n-(k-2))+(n-(k-3))+\ldots\ldots\ldots\ldots+n$

$= T(n-k)+ (n-k+1)+(n-k+2)+(n-k+3)+\ldots\ldots\ldots\ldots+n$

Assuming n==k

$=T(0)+1+2+3+\ldots\ldots\ldots+n$

$= 0+1+2+3+\ldots\ldots\ldots+n$

$=n(n+1)/2 = (n*n +n)/2 = n*n/2 +n/2 == O(n*n)$

$= O(n*n)=O(n^2)$

TG:sppu_te_be-comp

Recurrence Relation (Example)

2. Solve the following Recurrance Relations T(n) = T (n-1) +1 if T(0)= 0 **By Backward Substitution.**

Solution:

T(n)= T(n-1)+1 ----------1

Now here to compute T(n) we require T(n-1) so for that Replace n by n-1 in eq 1

Put n=n-1 in eq 1

T(n-1)=T(n-1-1)+1

=T(n-2)+1

Put back T(n-1) in eq 1 so it will become

T(n)=T(n-2)+1+1

T(n) =T(n-2)+2 ----------2

Now here to compute T(n) we require T(n-2) so for that Put n=n-2 in eq 1

TG:sppu_te_be-comp

Recurrence Relation (Example)

T(n-2)=T(n-2-1)+1

=T(n-3)+1

Put back value of T(n-2) in eq 2 so it will become

T(n)=T(n-3)+1+2

T(n)=T(n-3)+3

So the kth term will be

T(n)=T(n-k)+k

Assuming n==k

T(n)=T(0)+n Initial condition T(0)=0

T(n)= 0 +n

=O(n)

TG:sppu_te_be-comp

Recurrence Relation (Example)

Solve the following Recurrance Relations $T(n) = 2T(n/2) + n$ if $T(0) = 0$ **By Backward Substitution.**

Solution:

$T(n) = 2T(n/2)+n$ ----------1 when $T(1)=0$

Put $n=n/2$ in eq 1

$T(n/2) = 2T(n/2/2)+n/2$

$T(n/2) = 2T(n/4)+n/2$

Put back it in eq 1

$T(n)=2(2T(n/4)+n/2) +n = (4T(n/4)+n) +n$

$T(n)=4T(n/4)+2n$--------2

Replace n by n/4 in eq 1

$T(n/4) = 2T(n/4/2)+n/4$

TG:sppu_te_be-comp

Recurrence Relation (Example)

T(n/4)= 2T(n/8)+n/4 but back this in eq 2

T(n)=4(2T(n/8)+n/4)+2n

T(n)= (8T(n/8)+4n/4)+2n

T(n)= 8T(n/8)+3n

T(n)=2^3T(n/2^3)+3n

So the kth term will be

T(n)=2^kT(n/2^k)+kn-------3

Assume 2^k=n k=log2n

So eq 3 will becomes now

T(n)=nT(n/n)+log2n n

T(n)=nT(1)+nlog$_2$n since T(1)=0 n+nlog2n

T(n)=O(nlog2n)

TG.sppu_te_be-comp

Recurrence Relation

Masters Theorem: The Master Method is used for solving the following types of recurrence

$$T(n)=aT(n/b)+f(n) \quad \text{iff } a>=1 \ \& \ b>1$$

Where:

- n is the size of the problem.
- a is the number of subproblems in the recursion.
- n/b is the size of each subproblem. (Here it is assumed that all subproblems are essentially the same size.)
- f (n) is the sum of the work done outside the recursive calls, which includes the sum of dividing the problem and the sum of combining the solutions to the subproblems.
- It is not possible always bound the function according to the requirement, so we make three cases which will tell us what kind of bound we can apply on the function.

TE-sppu_te_be-comp

Recurrence Relation

It can be solved by any of the following case:

where, T(n) has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.

   OR { if $n^{\log_b a} > f(n)$ then $T(n) = \Theta(n^{\log_b a})$ }

2. If $f(n) = \Omega(n^{\log_b a + \epsilon})$, then $T(n) = \Theta(f(n))$.

   OR { if $n^{\log_b a} < f(n)$ then $T(n) = \Theta(f(n))$ }

3. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} * \log n)$.

   OR { if $n^{\log_b a} = f(n)$ then $T(n) = \Theta(n^{\log_b a} * \log n)$ }

$\epsilon > 0$ is a constant.

TG:sppu_te_be-comp

Recurrence Relation (Example)

1. Slove the Recurence relation using Masters therom T(n) = 8T(n/2) + n2

Solution:

- Compare given eqetion with T(n) = aT(n/b) + f(n) to find a & b
- Here, a = 8 & b=2 & f(n)=n^2
- Now find $n^{\log_b a} = n^{\log_2 8} = n^{\log_2 2^3} = n^3$
- Now as $n^{\log_b a} > f(n)$ this example is in case 1
- So according to case 2 T(n) = T(n) = $\Theta(n^{\log_b a})$
- $T(n) = \Theta(n^3)$.

TG:spnu_te_be-comp

Recurrence Relation (Example)

2. Slove the Recurence relation using Masters therom $T(n) = 2T(n/2) + n \log n$

Solution:

- Compare given eqetion with $T(n) = aT(n/b) + f(n)$ to find a & b
- Here, a = 2 & b=2 & f(n)=n log n
- Now find $n^{\log_b a} = n^{\log_2 2} = n$
- Now as $n^{\log_b a} < f(n)$ this example is in case 2
- So according to case 2 $T(n) = T(n) = \Theta(f(n))$
- $T(n) = \Theta(n \log n)$.

TG.spnu_te_be-comp

Recurrence Relation (Example)

3. Slove the Recurence relation using Masters therom $T(n) = 9T(n/3) + n^3$

- Compare given eqetion with $T(n) = aT(n/b) + f(n)$ to find a & b
- Here, $a = 9$ & $b=3$ & $f(n)= n^3$
- Now find $n^{\log_b a} = n^{\log_3 9} = n^{\log_3 3^2} = n^2$
- Now as $n^{\log_b a} < f(n)$ this example is in case 2
- So according to case 2 $T(n) = T(n) = \Theta(f(n))$
- $T(n) = \Theta(n^3)$.

TG:sppu_te_be-comp

Recurrence Relation (Example)

Slove the Recurence relation using any suitable example.

1. T(n)=T(n/2) + 1
2. T(n)=2T(n/2)+C when T(1)=1
3. T(n)= T(n/3)+C when T(1)=1

TG:sppu_te_be-comp

P-NP Class problems

- **P Class Problems: The problems which can be solved in polynomial time of complexity are called P class problems.**
- **Ex. O (n), O(n^2), O(nlogn)**
- **Ex.**
  1. **Calculating the greatest common divisor.**
  2. **Searching & Sorting Algorithms**
  3. **Finding a maximum matching.**
  4. **Decision versions of linear programming.**
- The Problems than can be solved in polynomial time are called tractable problems.

TG:sppu_te_be-comp

P-NP Class problems

- **Features of P Class Problems**
- The solution to P problems is easy to find.
- P is often a class of computational problems that are solvable and tractable. Tractable means that the problems can be solved in theory as well as in practice. But the problems that can be solved in theory but not in practice are known as intractable.

TG:sppu_te_be-comp

P-NP Class problems

- **NP Class Problems:** The NP in NP class stands for **Non-deterministic Polynomial Time**. It is the collection of decision problems that can be solved by a non-deterministic machine in polynomial time.
- **Ex. O (2^n), O(3^n),**
- **Ex.**
  1. **Traveling Salesman's Problem,**
  2. **0/1 Knapsack problems**

TG:sppu_te_be-comp

P-NP Class problems

- **Features of NP Class Problems**
- The NP class Problems are also called as intractable problems.
- The solutions of the NP class are hard to find since they are being solved by a non-deterministic machine but the solutions are easy to verify.
- Problems of NP can be verified by a Turing machine in polynomial time

TG:sppu_te_be_comp

P-NP Class problems

	8			1			2	
6			3		5			1
		7				4		
	2		1		9		5	
7								6
	9		6		3		4	
		5				3		
9			2		1			8
	3			6			7	

- Consider an algorithm to solve Sudoku problem
- For every blank space having the option 1 to 9
- And have to fill approximately 50 empty boxes
- So complexity is 9^{50}
- So is not a P class problem.

TG:sppu_te_be-comp

P-NP Class problems

3	8	9	7	1	4	6	2	5
6	4	2	3	9	5	7	8	1
5	1	7	8	2	6	4	3	9
4	2	6	1	7	9	8	5	3
7	5	3	4	8	2	1	9	6
1	9	8	6	5	3	2	4	7
8	6	5	9	4	7	3	1	2
9	7	4	2	3	1	5	6	8
2	3	1	5	6	8	9	7	4

- But if solution is given and we just want to verify solution whether is correct or incorrect then it can be done in a polynomial time.
- So we can verify the problems solution in polynomial time but cannot solve in polynomial time

TG:sppu_te_be-comp

P Vs NP Class problems

- To understand the relation between P & NP Class problems consider the following three cases

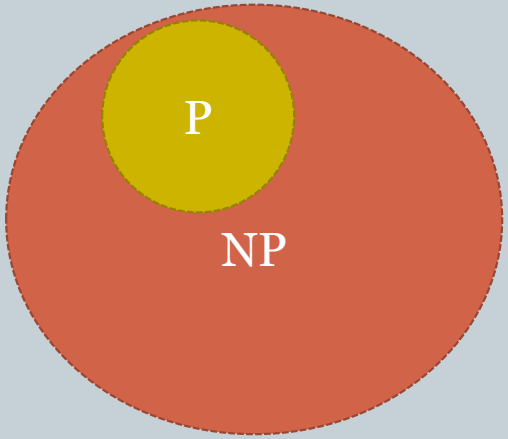
1: If P==NP Which means

- **Every problem can be solvable in polynomial Time which is not actually feasible.**

2: If P!=NP Which means

- **Every problem cannot be solvable in polynomial Time which is not again actually feasible.**
- **So What is the relation between P & NP**

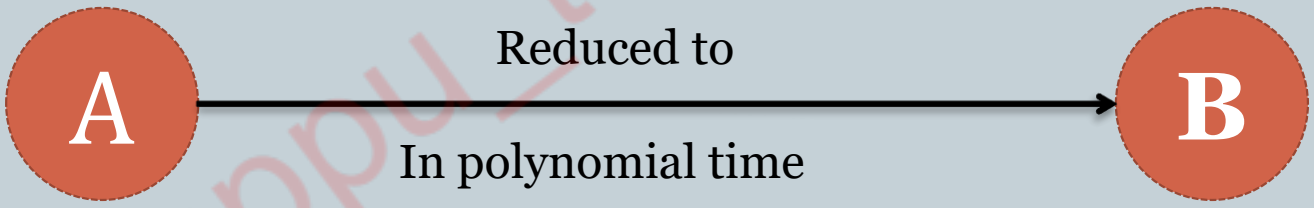
$$P \subseteq NP$$



TG.sppu_te_be-comp

Reduction

- If A problem is reduced to B Problem in polynomial time then is called reduction.
- In simple word, problem A is **reducible** to problem B if an algorithm for solving problem A (if it existed) efficiently could also be used as a subroutine to solve problem A efficiently.



Let A & B are two NP problems, And problem A is Reduced to problem B , If there is a way to solve A by Non deterministic polynomial algorithm that solve B also in polynomial time So B is always in polynomial time.

TG:sppu_te_be-comp

Reduction

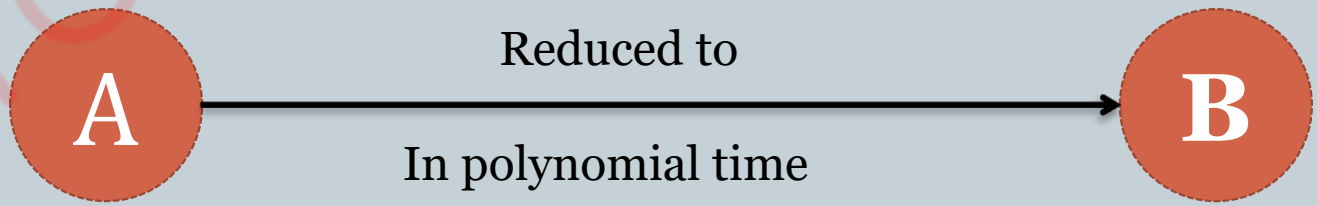
- Let us Understand

1. **Traveling Salesman's Problem $O(2^n)$**
2. **0/1 Knapsack problems $O(2^n)$**
3. **Sum of Subset Problem $O(2^n)$**
4. **Graph Coloring Problems $O(2^n)$**
5. **Hamilton Cycle $O(2^n)$**

Here A is Base Problem

And B is any of the problem from the List

- If problem A is reduced to B and if B in NP then A is also in NP.



TG:sppu_te_be-comp

NP Hard Problem

A problem is NP-hard if an algorithm for solving it can be translated into one for solving any NP-problem (nondeterministic polynomial time) problem.

Some of the examples of problems in Np-hard are:

1. Halting problem.
2. Qualified Boolean formulas.
3. No Hamiltonian cycle.

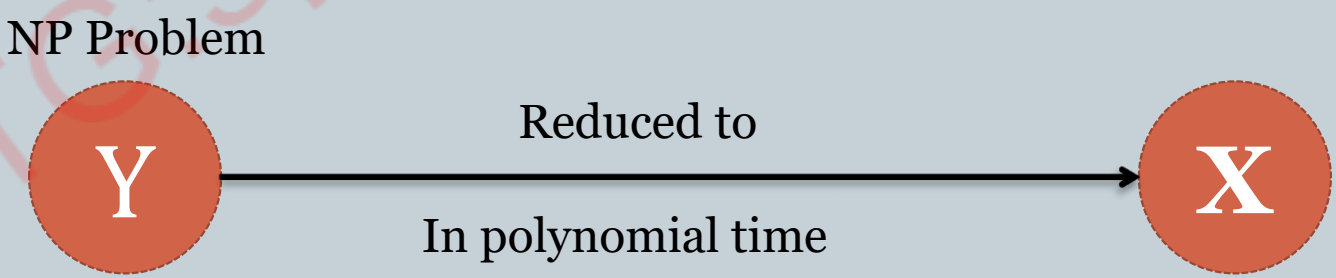
TG:sppu_te_be-comp

NP Complete Problem

A problem X is NP-Complete if there is an NP problem Y, such that Y is reducible to X in polynomial time. NP-Complete problems are as hard as NP problems. A problem is NP-Complete if it is a part of both NP and NP-Hard Problem

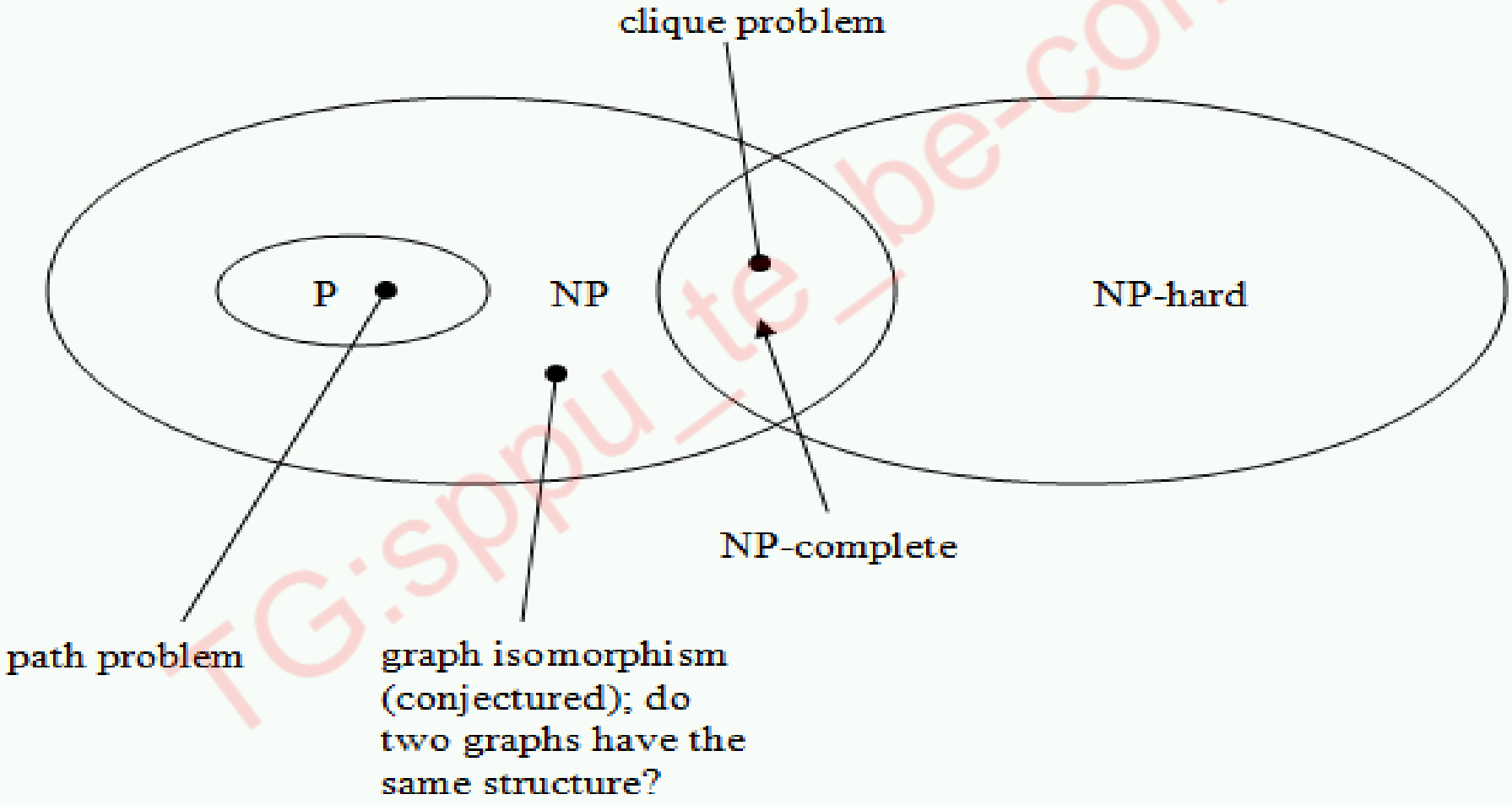
Some of the examples of problems in Complete are:

1. Decision version of 0/1 Knapsack.
2. Hamiltonian Cycle.
3. Satisfiability.
4. Vertex cover.



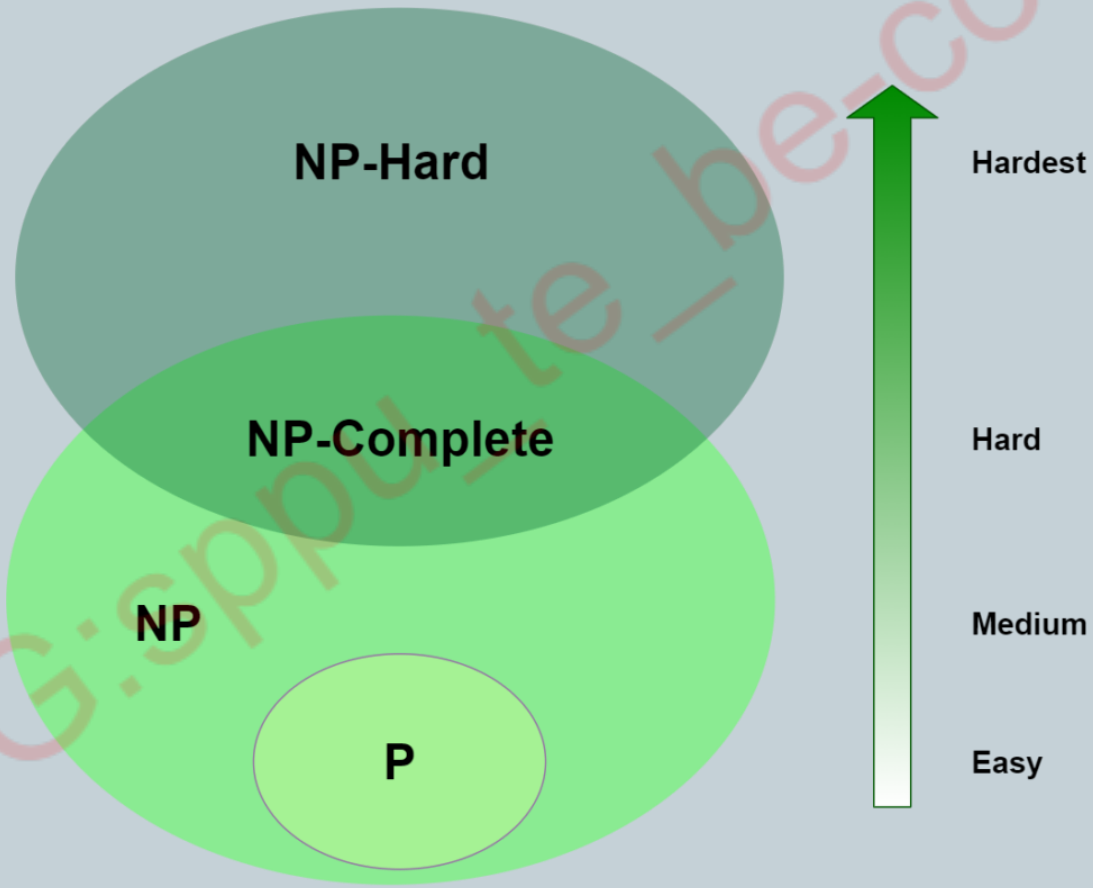
TG:sppu_te_be-comp

Relation Between P, NP, NP Hard & NP Complete



TG:sppu_te_be-comp

Relation Between P, NP, NP Hard & NP Complete



TG:sppu_te_be-comp

Deterministic Vs Non Deterministic Algorithms

NP HARD	NP COMPLETE
NP-Hard problems(say X) can be solved if and only if there is a NP-Complete problem(say Y) that can be reducible into X in polynomial time.	NP-Complete problems can be solved by a non-deterministic Algorithm /Turing Machine in polynomial time.
To solve this problem, it do not have to be in NP .	To solve this problem, it must be both NP and NP-hard problems.
Do not have to be a Decision problem.	It is exclusively a Decision problem.
Ex. Halting problem.	Ex. Hamiltonian Cycle.

TC:sppu_te_be-comp

Deterministic Algorithms

- In computer science, a **deterministic algorithm** is an **algorithm** which, given a particular input, will always produce the same output, with the underlying machine always passing through the same sequence of states.
- The algorithms in which the result of every algorithm is uniquely defined are known as the **deterministic algorithm**.

TG:sppu_te_be_comp

Deterministic Algorithms

- **Some of the terms related to the non-deterministic algorithm are defined below**:
- **choice(X) :** chooses any value randomly from the set X.
- **failure() :** denotes the unsuccessful solution.
- **success() :** Solution is successful and current thread terminates.

TG:sppu_te_be-comp

Deterministic Algorithms

- ***Problem Statement :*** *Search an element x on A[1:n] where n>=1, on successful search return j if a[j] is equals to x otherwise return 0.*
- ***Non-deterministic Algorithm for this problem :***

```
j= choice(a, n)
if(A[j]==x) then
{
    write(j);
    success();
}
write(0);
failure();
```

TG:sppu_te_be-comp

Deterministic Vs Non Deterministic Algorithms

DETERMINISTIC ALGORITHM	NON-DETERMINISTIC ALGORITHM
For a particular input the computer will give always same output.	For a particular input the computer will give different output on different execution.
Can solve the problem in polynomial time.	Can't solve the problem in polynomial time.
Can determine the next step of execution.	Cannot determine the next step of execution due to more than one path the algorithm can take.

TG:sppu_te_be-comp

Vertex Cover

- A vertex cover of an undirected graph is a subset of its vertices such that for every edge (u, v) of the graph, either 'u' or 'v' is in the vertex cover. Although the name is Vertex Cover, the set covers all edges of the given graph. ***Given an undirected graph, the vertex cover problem is to find minimum size vertex cover***.
- The vertex Cover of a graph is defined as a subset of its vertices, such for every edge in the graph, from vertex **u** to **v,** at least one of them must be a part of the vertex cover set.

TG.sppu_te_be-comp

Vertex Cover

- vertex cover problem are the approximation algorithms that run in polynomial time complexity. A simple approximate algorithm for the vertex cover problem is described below:

- Initialize the vertex cover set as empty.
- Let the set of all edges in the graph be called E.
- While E is not empty:
- Pick a random edge from the set E, add its constituent nodes, u and v into the vertex cover set.
- For all the edges, which have either u or v as their part, remove them from the set E.
- Return the final obtained vertex cover set, after the set E is empty.

TG:sppu_te_be-comp

Vertex Cover

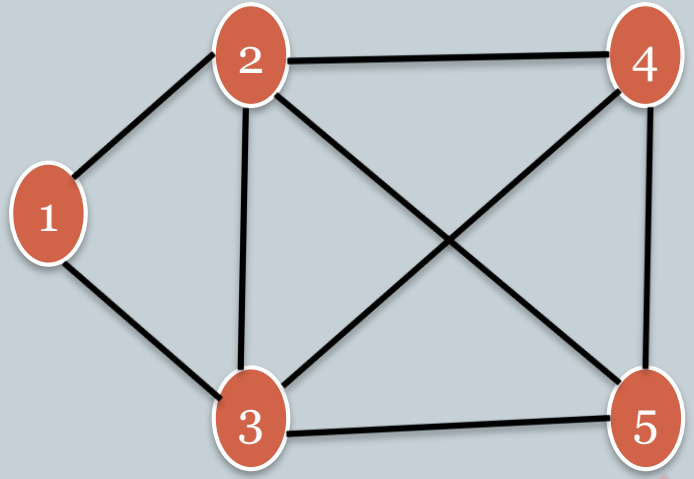

```
def greedy (E, V)
C = { }
while E not empty:

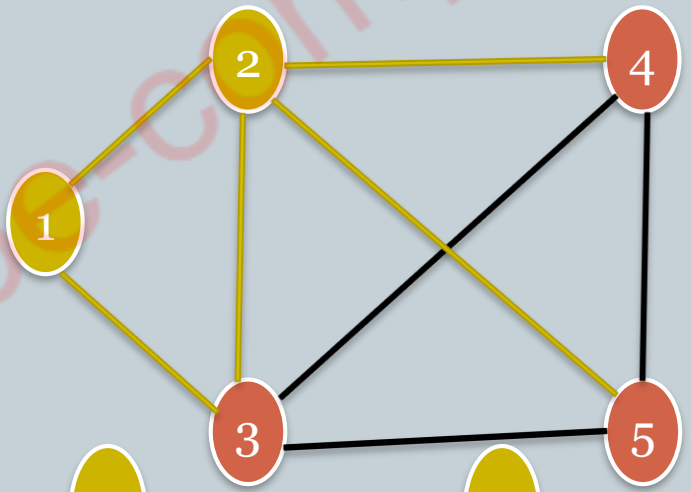
        select any edge with endpoints (u, v) from E
        add ( u, v ) to C
        remove all edges incident to u or v from E

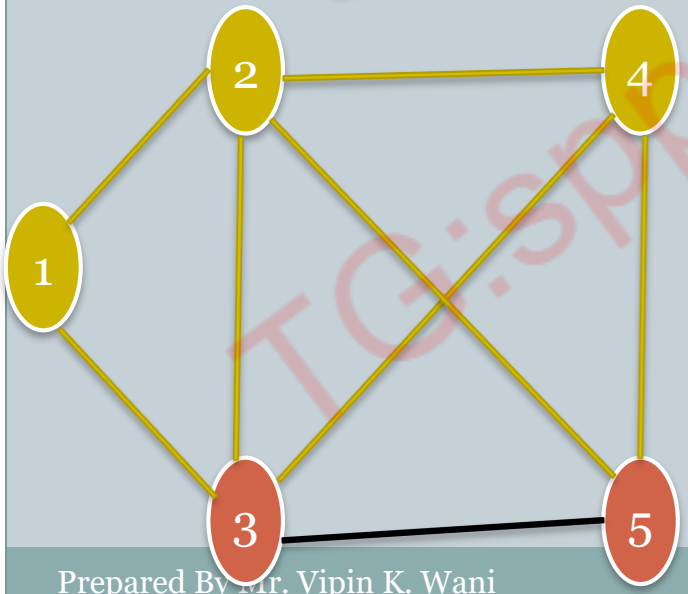
return C
```

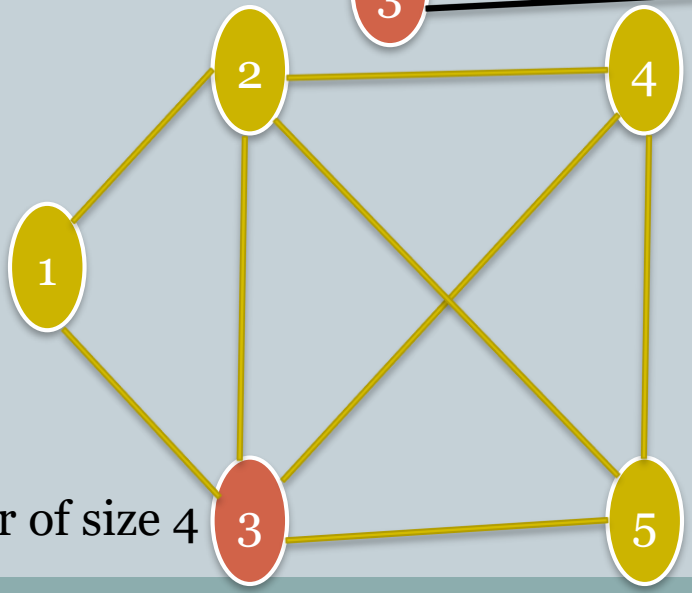
TG:spp_te_be-comp

Vertex Cover





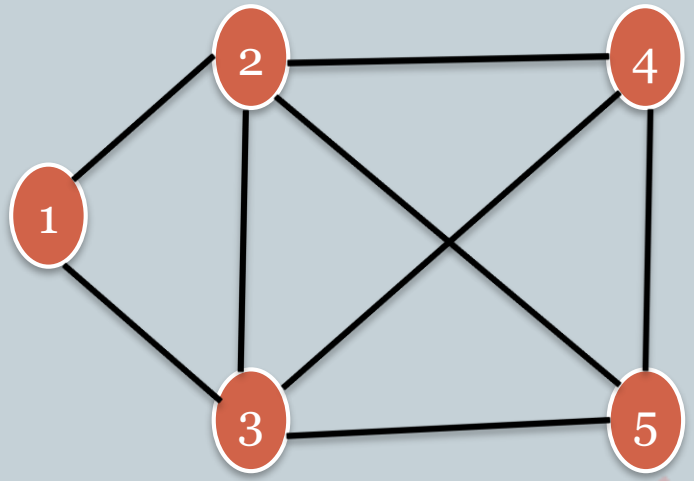


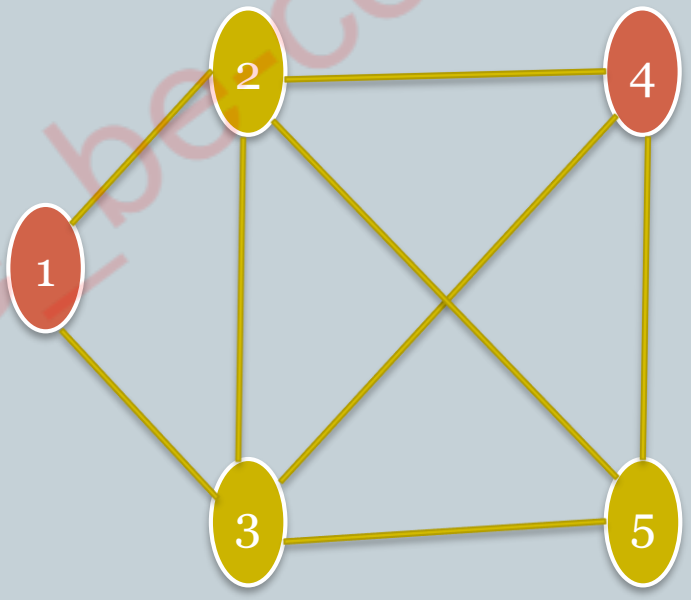


Vector Cover of size 4

TG:sppu_te_be-comp

Vertex Cover





Vector Cover of size 3

TG:sppu_te_becomp

Hamilton Cycle

Hamiltonian Path in an undirected graph is a path that visits each vertex exactly once. A Hamiltonian cycle (or Hamiltonian circuit) is a Hamiltonian Path such that there is an edge (in the graph) from the last vertex to the first vertex of the Hamiltonian Path. Determine whether a given graph contains Hamiltonian Cycle or not. If it contains, then prints the path. Following are the input and output of the required function.

Any Hamiltonian Path can be made into a Hamiltonian Circuit through a polynomial time reduction by simply adding one edge between the first and last point in the path. Therefore we have a reduction, which means that **Hamiltonian Paths are in NP Hard**, and therefore in NP Complete.

TE-sppu_te_be-comp

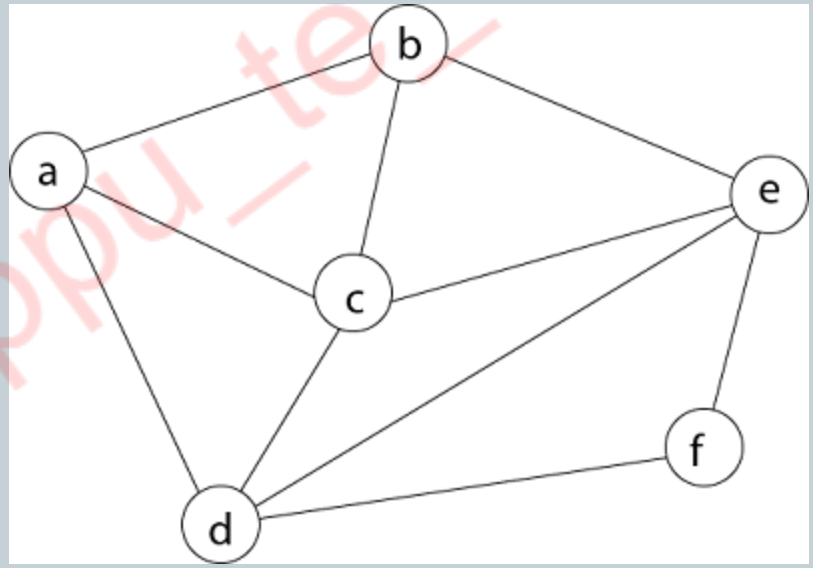
Steps to find Hamilton Cycle (Backtracking approach)

- Given a graph G = (V, E) we have to find the Hamiltonian Circuit using.
- We start our search from any arbitrary vertex say 'a.' This vertex 'a' becomes the root of our implicit tree.
- The first element of our partial solution is the first intermediate vertex of the Hamiltonian Cycle that is to be constructed.
- The next adjacent vertex is selected by alphabetical order.
- If at any stage any arbitrary vertex makes a cycle with any vertex other than vertex 'a' then we say that **dead end** is reached.
- In this case, we backtrack one step, and again the search begins by selecting another vertex and backtrack the element from the partial; solution must be removed.
- The search using backtracking is successful if a Hamiltonian Cycle is obtained.

TG:sppu_te_be-comp

Hamilton Cycle

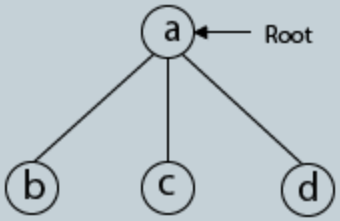
Example: Consider a graph G = (V, E) shown in fig. we have to find a Hamiltonian circuit using Backtracking method.

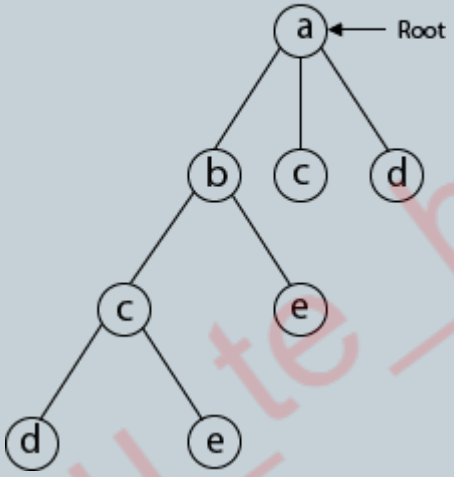


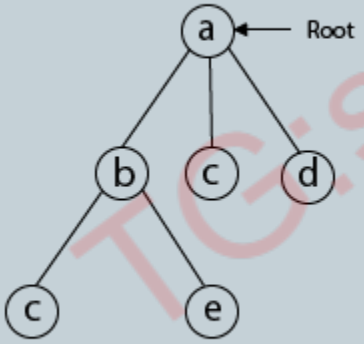
TG:sppu_te_be-comp

Hamilton Cycle

Example:



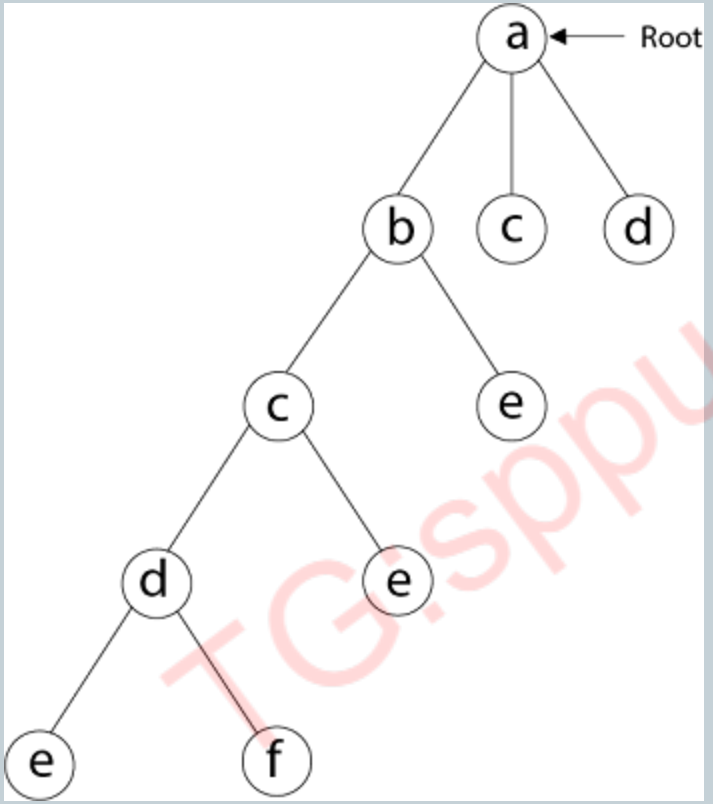


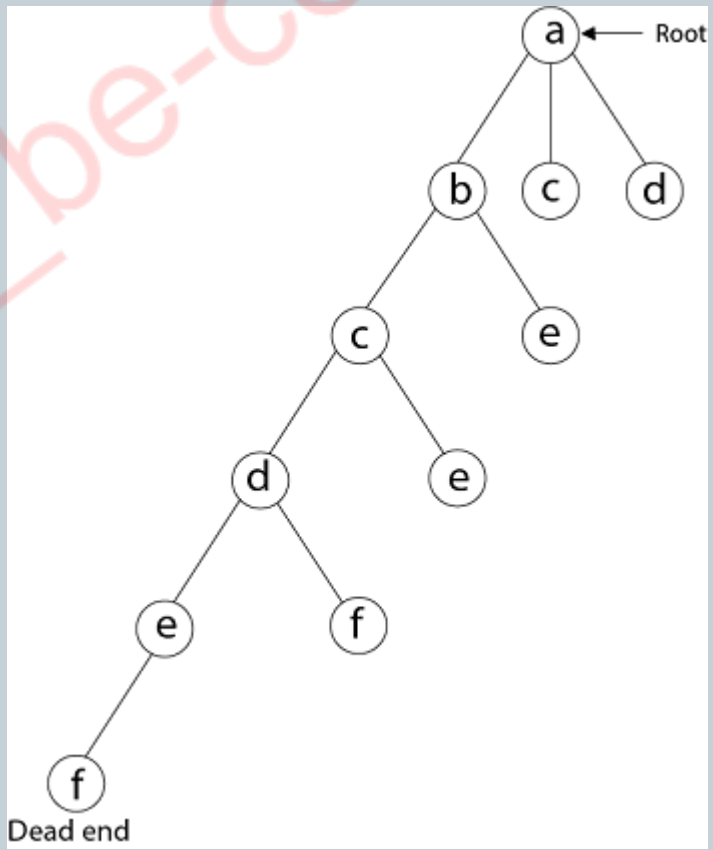


TE:sppu_te_be-comp

Hamilton Cycle

Example:

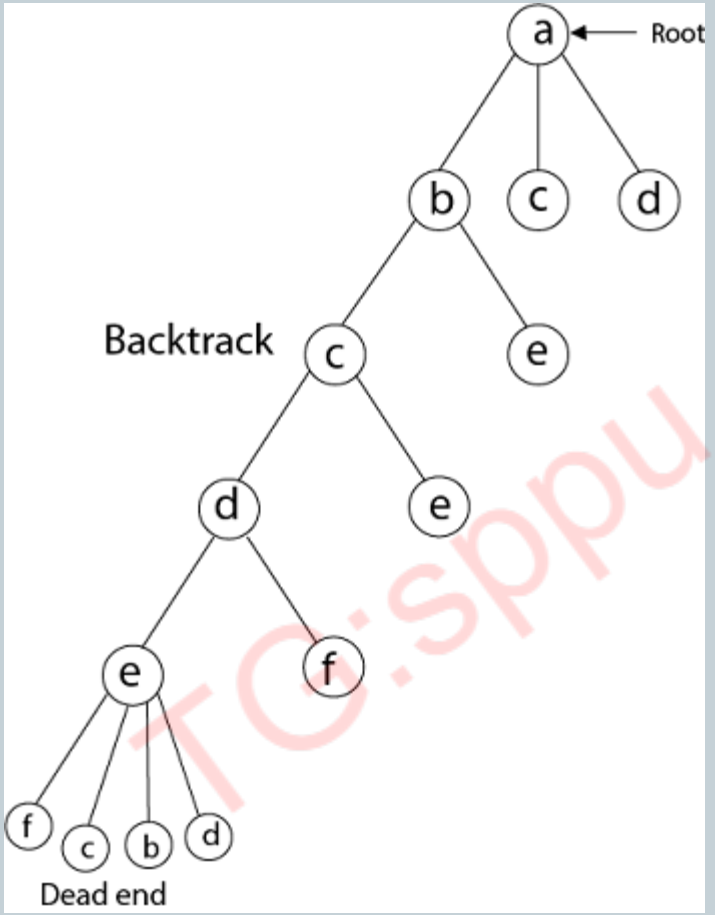


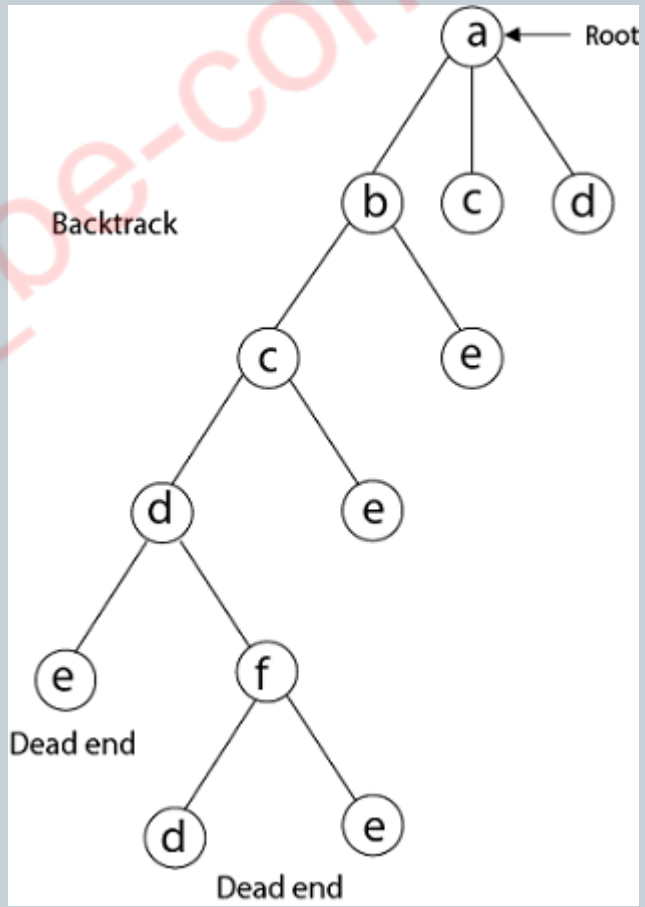


TG:sppu_te_be-comp

Hamilton Cycle

Example:

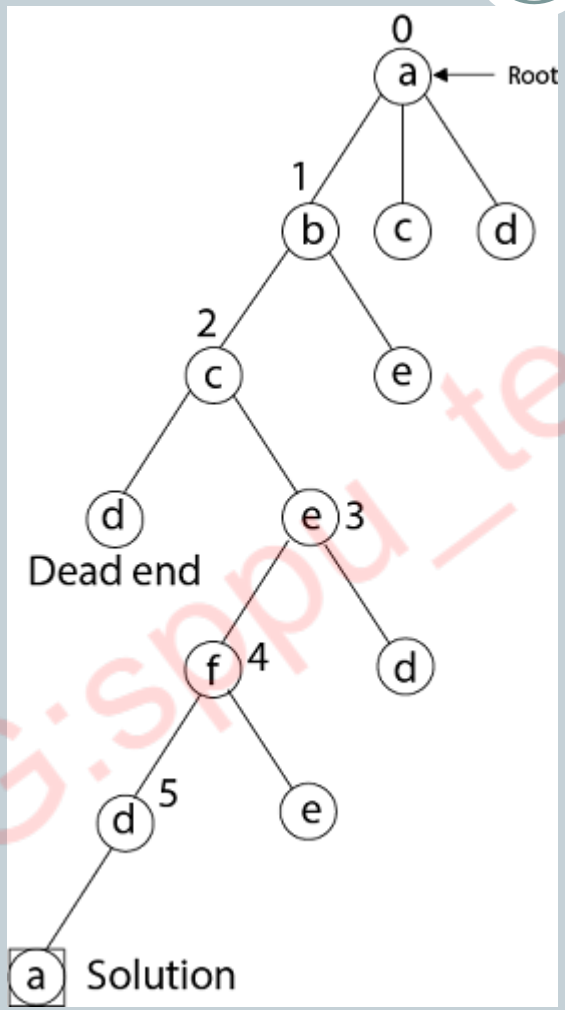




TG:sppu_te_be-comp

Hamilton Cycle

Example:



TG:sppu_te_be-comp

Satisfiability problem (3 SAT)

- The satisfiability problem, usually called SAT, is the following language. SAT = {φ | φ is a satisfiable clausal formula}.
- Thought of as a computational problem, the input to SAT is a clausal formula φ and the problem is to determine whether φ is satisfiable.
- Solving a 3-SAT problem is the act of finding a set of variable assignments to True or False that make that statement true or alternately providing a proof.
- **Boolean satisfiability problem** also called **propositional satisfiability problem** and abbreviated **SATISFIABILITY**, **SAT** is the problem of determining if there exists an interpretation that satisfies a given Boolean formula.
- In other words, it asks whether the variables of a given Boolean formula can be consistently replaced by the values TRUE or FALSE in such a way that the formula evaluates to TRUE. If this is the case, the formula is called *satisfiable*.

TG:spul_te_be-comp

Satisfiability problem (3 SAT)

- Boolean, or propositional-logic expressions are built from variables and constants using the operators AND, OR, and NOT.
- Constants are true and false, represented by 1 and 0, respectively.
- We'll use concatenation for AND, + for OR, - for NOT, unlike the text.
- Example:
- Consider three Boolean variables y_1, y_2, and y_3.
- Consider clausal formula $F = (y_1 \vee \hat{y}_2 \vee y_3) \wedge (\hat{y}_1 \vee y_2 \vee \hat{y}_3)$
- So the objective of the problem is then to decide whether *F* is *satisfiable*, *i.e.*, whether there exists an assignment *a* of truth values *true* and *false* to the variables y_k such that every clause contains at least one literal rendered true by *a*.

TG:spoul_te_be-comp

Satisfiability problem (3 SAT)

- Let us Consider different possible values of y_1, y_2 & y_3

y1	y2	y3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

So let us chech for which combination of y1, y2 & y3 F is satisfied

TG:sppu_te_becomp

Satisfiability problem (3 SAT)

- Let us Consider different possible values of y_1, y_2 & y_3
- So let us chech for which combination of y1, y2 & y3 F is satisfied
- If y1=0, y2=0 & y3=0

F= $(y_1 \vee \hat{y}_2 \vee y_3) \wedge (\hat{y}_1 \vee y_2 \vee \hat{y}_3) = (0 \vee 1 \vee 0) \wedge (1 \vee 0 \vee 1) = 1 \wedge 1 = 1$ Which is true

- If y1=0, y2=0 & y3=1

F= $(y_1 \vee \hat{y}_2 \vee y_3) \wedge (\hat{y}_1 \vee y_2 \vee \hat{y}_3) = (0 \vee 1 \vee 1) \wedge (1 \vee 0 \vee 0) = 1 \wedge 1 = 1$ Which is true

…

So like this we will check for each combination of input.

So to check the satisfiability of F for all 8 possible combination it will take O(8) i.e. $O(2^3)$ time.

So for n Boolean variable it would take $O(2^n)$ time to verify satisfiability of F.

TG:sppu_te_be-comp

Thank You...!

